

Minimum Insertions to Balance a Parentheses String

Language: Java

Question

Given a string **s** containing only '(' and ')', find the minimum number of parentheses characters that must be inserted to make the string balanced. Every '(' must be matched with two consecutive closing parentheses '))', and the opening parenthesis must appear before its matching pair.

Example

Input: s = "(()))"

Output: 1

Explanation: Insert one ")" at the end to make the string balanced: "(())))".

Java Solution

```
class Solution {
    public int minInsertions(String s) {
        int insertions = 0;
        int open = 0;

        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);

            if (c == '(') {
                open++;
            } else {
                if (i + 1 < s.length() && s.charAt(i + 1) == ')') {
                    i++;
                } else {
                    insertions++;
                }

                if (open > 0) {
                    open--;
                } else {
                    insertions++;
                }
            }
        }

        insertions += open * 2;
        return insertions;
    }
}
```

Simple Logic

1. When you see '(', increase **open**.
2. A closing pair '))' matches one opening '('.
3. If a ')' has no second ')' beside it, count one insertion for the missing ')'.
4. If a closing pair has no unmatched '(', count one insertion for a missing '('.
5. At the end, every unmatched '(' needs two ')' characters.

Complexity

Time: O(n)

Extra space: O(1)